📚 TÀI LIỆU TOÀN DIỆN CHI TIẾT TẤT CẢ API & CÔNG NGHỆ HỆ THỐNG E-COMMERCE MICROSERVICES

Dự án: E-Commerce Microservices System

Phiên bản: 2.5.0 Enterprise Edition

Ngày cập nhật: 15/09/2026

Tác giả: Antigravity AI Assistant & Engineering Team

Tổng số Microservices: 13 Microservices Backend + 1 API Gateway + 1 Eureka Discovery Server + Next.js 16 BFF Web App

---

📑 MỤC LỤC CHI TIẾT HỆ THỐNG

1. [Chương 1: Kiến trúc Hệ thống & Nguyên lý Thiết kế API](#chương-1-kiến-trúc-hệ-thống--nguyên-lý-thiết-kế-api)
2. [Chương 2: Chuẩn Xác thực, Phân quyền & Mã trạng thái HTTP](#chương-2-chuẩn-xác-thực-phân-quyền--mã-trạng-thái-http)
3. [Chương 3: Chi tiết Dịch vụ User Service (`user-service`)](#chương-3-chi-tiết-dịch-vụ-user-service-user-service)
4. [Chương 4: Chi tiết Dịch vụ Product Catalog Service (`product-catalog-service`)](#chương-4-chi-tiết-dịch-vụ-product-catalog-service-product-catalog-service)
5. [Chương 5: Chi tiết Dịch vụ Inventory Service (`inventory-service`)](#chương-5-chi-tiết-dịch-vụ-inventory-service-inventory-service)
6. [Chương 6: Chi tiết Dịch vụ Cart Service (`cart-service`)](#chương-6-chi-tiết-dịch-vụ-cart-service-cart-service)
7. [Chương 7: Chi tiết Dịch vụ Order Service (`order-service`)](#chương-7-chi-tiết-dịch-vụ-order-service-order-service)
8. [Chương 8: Chi tiết Dịch vụ Payment Service (`payment-service`)](#chương-8-chi-tiết-dịch-vụ-payment-service-payment-service)
9. [Chương 9: Chi tiết Dịch vụ Sale / Voucher Service (`sale-service`)](#chương-9-chi-tiết-dịch-vụ-sale--voucher-service-sale-service)
10. [Chương 10: Chi tiết Dịch vụ Review Service (`review-service`)](#chương-10-chi-tiết-dịch-vụ-review-service-review-service)
11. [Chương 11: Chi tiết Dịch vụ AI Chatbot Service (`ai-chatbot-service`)](#chương-11-chi-tiết-dịch-vụ-ai-chatbot-service-ai-chatbot-service)

12. [Chương 12: Chi tiết Dịch vụ Product Recommendation Service (`product-recommendation-service`)](#chương-12-chi-tiết-dịch-vụ-product-recommendation-service-product-recommendation-service)
13. [Chương 13: Chi tiết Dịch vụ Activity Log Service (`activity-log-service`)](#chương-13-chi-tiết-dịch-vụ-activity-log-service-activity-log-service)
14. [Chương 14: Chi tiết Dịch vụ Notification Service (`notification-service`)](#chương-14-chi-tiết-dịch-vụ-notification-service-notification-service)
15. [Chương 15: Chi tiết Dịch vụ Telegram Service (`telegram-service`)](#chương-15-chi-tiết-dịch-vụ-telegram-service-telegram-service)
16. [Chương 16: Chi tiết Tầng API Gateway & Route Config (`api-gateway`)](#chương-16-chi-tiết-tầng-api-gateway--route-config-api-gateway)
17. [Chương 17: Chi tiết Tầng Next.js BFF API Routes (`stitch/app/api`)](#chương-17-chi-tiết-tầng-nextjs-bff-api-routes-stitchappapi)
18. [Chương 18: Bảng Thống kê Toàn bộ Công nghệ & Thư viện đang sử dụng](#chương-18-bảng-thống-kê-toàn-bộ-công-nghệ--thư-viện-đang-sử-dụng)
19. [Chương 19: Quy trình Triển khai & Vận hành (Deployment & Operations)](#chương-19-quy-trình-triển-khai--vận-hành-deployment--operations)

CHƯƠNG 1: KIẾN TRÚC HỆ THỐNG & NGUYÊN LÝ THIẾT KẾ API

1.1 Tổng quan Mô hình Microservices Phân tán

Hệ thống E-Commerce của bạn được kiến trúc dựa trên nền tảng Microservices hoàn chỉnh, cho phép mở rộng độc lập từng thành phần (Independent Scalability), cách ly sự cố (Fault Isolation) và phát triển/triển khai liên tục (CI/CD).

Các thành phần hạ tầng cốt lõi:

1. **Client Layer (Web & Mobile Clients)**: Giao diện người dùng xây dựng bằng Next.js 16 App Router, tương tác API qua cổng Gateway duy nhất.

2. **API Gateway (`api-gateway` - Port 8080)**: Cổng tập trung điều hướng toàn bộ lưu lượng mạng (Traffic). Đảm nhận:

- Reverse Proxy & Routing: Chuyển tiếp Request tới đúng Microservice backend tương ứng dựa trên URL Prefix.
- CORS Management: Cấu hình Cross-Origin tập trung cho tất cả domain phía Client.
- Authentication & JWT Filter: Giải mã & Kiểm tra chữ ký số của JWT Token trước khi cho phép đi sâu vào nội bộ.
- Rate Limiting: Giới hạn số lượng request per second chống tấn công từ chối dịch vụ DoS/DDoS.

3. **Service Discovery (`eureka-server` - Port 8761)**: Netflix Eureka Server giữ vai trò danh bạ định danh dịch vụ động. Khi một service khởi chạy, nó gửi Heartbeat đăng ký IP/Port với Eureka. Các service khác truy vấn nhau qua Service ID mà không cần cấu hình IP cứng.

4. **Backend Services Layer**: Tập hợp 13 microservices chuyên biệt phụ trách logic nghiệp vụ domain theo mô hình Database-per-Service (mỗi service quản lý schema CSDL độc lập).

1.2 Nguyên lý Nhất quán Dữ liệu & Giao dịch Phân tán (Distributed Transactions)

Trong hệ thống Microservices phân tán, không thể sử dụng giao dịch ACID truyền thống xuyên suốt các database. Hệ thống áp dụng **Mô hình Saga dựa trên giao dịch bù trừ (Compensating Transactions)**:

- **Quy trình Đặt hàng & Giữ kho (Inventory Reservation with TTL)**:

1. Khi người dùng gửi yêu cầu Checkout (`POST /orders/checkout`), `order-service` gọi `inventory-service` để khóa/giữ số lượng tồn tạm thời (Reserved Stock) thay vì trừ trực tiếp tồn kho vĩnh viễn.
2. Đồng thời gọi `sale-service` để kiểm tra và tạm khóa mã giảm giá (Voucher).
3. Đơn hàng được khởi tạo ở trạng thái `PENDING_PAYMENT` với thời gian chờ thanh toán (TTL 15 phút).

- **Cơ chế Bù trừ (Compensation)**:
- Nếu thanh toán thành công (qua Webhook SePay/PayOS/VNPay), trạng thái chuyển sang `CONFIRMED` và xác nhận trừ kho thực tế.
- Nếu hết thời gian chờ thanh toán hoặc người dùng chủ động hủy (`POST /orders/{id}/cancel`), hệ thống kích hoạt giao dịch bù trừ: tự động hoàn trả số lượng hàng về kho khả dụng và mở khóa voucher.

1.3 Cơ chế Chịu lỗi & Chống sụp đổ dây chuyền (Fault Tolerance & Circuit Breaker)

- Để tránh hiện tượng **Cascade Failure** (sụp đổ dây chuyền khi một service con bị nghẽn hoặc chết), các kết nối OpenFeign giữa các service được trang bị cơ chế giới hạn thời gian (Connect/Read Timeout) và Fallback.
- Các dịch vụ phụ trợ như `activity-log-service`, `product-recommendation-service`, `telegram-service` được thiết kế theo nguyên lý non-blocking / degraded gracefully: nếu dịch vụ gợi ý hoặc ghi log tạm thời gián đoạn, luồng mua hàng và tra cứu sản phẩm chính vẫn hoạt động trơn tru mà không làm gián đoạn trải nghiệm của khách hàng.

CHƯƠNG 2: CHUẨN XÁC THỰC, PHÂN QUYỀN & MÃ TRẠNG THÁI HTTP

2.1 Chuẩn giao thức xác thực JWT (JSON Web Token)

- **Token Format**: Chữ ký HMAC SHA-256 (`HS256`).
- **Access Token Header**: `Authorization: Bearer <access_token>`
- **Thời gian sống Access Token**: `24 giờ` (86,400,000 ms).
- **Thời gian sống Refresh Token**: `7 ngày` (604,800,000 ms).

2.2 Bảng Mã Trạng Thái HTTP (HTTP Status Codes)

Mã lỗi HTTP	Tên chuẩn	Ý nghĩa & Trường hợp áp dụng
200	OK	Thao tác thành công, trả về dữ liệu chuẩn.
201	Created	Tạo mới dữ liệu thành công (Đơn

		hàng, User, Sản phẩm).
400	Bad Request	Dữ liệu gửi lên sai cú pháp, thiếu tham số bắt buộc.
401	Unauthorized	Thiếu Token xác thực hoặc Token không hợp lệ.
403	Forbidden	Người dùng không đủ quyền truy cập.
404	Not Found	Không tìm thấy tài nguyên trong cơ sở dữ liệu.
423	Locked	Tài khoản bị tạm khóa do nhập sai mật khẩu quá 5 lần.
428	Precondition Required	Yêu cầu bước xác thực nâng cao (Xác thực OTP Email hoặc 2FA).
500	Internal Server Error	Lỗi không mong muốn tại phía Server Backend.

2.3 Cơ chế Thu hồi Token & Vô hiệu hóa Phiên tức thì (Token Invalidation Mechanism)

- Mặc dù JWT là Stateless Token, nhưng hệ thống vẫn đảm bảo an toàn tuyệt đối khi có biến cố bảo mật:
- **Khi đổi mật khẩu (`PUT /users/me/change-password`)**: Token version hoặc thời điểm cập nhật mật khẩu (`password_changed_at`) được ghi nhận. API Gateway kiểm tra mốc thời gian phát hành token (`iat`), nếu nhỏ hơn thời điểm đổi mật khẩu thì từ chối ngay lập tức.
- **Khi tài khoản bị khóa (`PUT /admin/users/{id}/unlock`)**: Trạng thái người dùng được đồng bộ và kiểm tra tại Gateway Filter; người dùng bị khóa sẽ bị chặn truy cập ngay lập tức dù Access Token vẫn còn hạn 24 giờ.

2.4 Kiến trúc Phòng chống Lỗ hổng IDOR (Insecure Direct Object Reference)

Lỗ hổng IDOR xảy ra khi người dùng thay đổi các ID tham chiếu (như `userId`, `orderId`, `cartId`) để truy cập hoặc chỉnh sửa trái phép dữ liệu của người khác. Hệ thống triển khai giải pháp phòng thủ đa tầng:

1. **Trích xuất Danh tính Từ Context Tin cậy (Zero-Trust Identity Extraction)**:

- Tất cả các API người dùng (như `/cart/`, **`/orders/my-orders`, `/users/me`)** tuyệt đối không** nhận `userId` qua Request Body hoặc Query Param từ Client.
- API Gateway sau khi xác thực JWT sẽ trích xuất `userId` và `role` từ Payload đã ký số, sau đó chuyển tiếp qua Header nội bộ tin cậy `X-User-Id` và `X-User-Roles` vào các microservice.

2. **Kiểm tra Quyền Sở hữu Dữ liệu (Object Ownership Verification)**:

- Khi truy vấn tài nguyên cụ thể (Ví dụ: `GET /orders/{id}` hoặc `POST /orders/{id}/cancel`), tầng Service luôn thực hiện kiểm tra chéo:

$$\text{order.getUserId().equals(currentAuthenticatedUserId) } \lor \text{ currentUser.hasRole("ROLE_ADMIN")}$$

- Nếu ID không trùng khớp, hệ thống lập tức chặn và trả về mã lỗi `403 Forbidden` thay vì thực hiện truy vấn.

3. **Phân quyền dựa trên vai trò (RBAC - Role-Based Access Control)**:

- Các API quản trị (`/admin/**`) được bảo vệ nghiêm ngặt bằng `@PreAuthorize("hasRole('ROLE_ADMIN')")` và được Gateway chặn từ vòng ngoài nếu JWT không chứa quyền tương ứng.

2.5 Cơ chế Phòng chống Lỗ hổng SQL Injection Toàn diện

Tất cả các tầng xử lý dữ liệu backend đều tuân thủ các nguyên tắc bảo mật cơ sở dữ liệu nghiêm ngặt:

1. **100% Parameterized Queries qua Spring Data JPA & Hibernate ORM**:

- Mọi câu lệnh truy vấn dữ liệu đều sử dụng kỹ thuật Prepared Statements với tham số có ràng buộc kiểu dữ liệu rõ ràng (Binding Parameters).
- Tuyệt đối nghiêm cấm việc cộng chuỗi thô (Raw String Concatenation) trong câu lệnh SQL hoặc JPQL.

2. **Kiểm soát Truy vấn Động & Tìm kiếm (Dynamic Criteria API & Specification)**:

- Các chức năng tìm kiếm sản phẩm phức tạp với nhiều bộ lọc (`keyword`, `minPrice`, `maxPrice`, `brandId`, `categoryId`) được xây dựng hoàn toàn bằng `JpaSpecificationExecutor` và `CriteriaBuilder`. Mọi chuỗi keyword đầu vào đều được Escape ký tự đặc biệt (`%`, `_`, `'`, `\`).

3. **Phòng thủ Ngay từ Cửa ngõ (Input Validation & Whitelisting)**:

- Áp dụng chặt chẽ Hibernate Validator (`@Valid`, `@NotBlank`, `@Pattern`, `@Size`, `@Min`, `@Max`) trên tất cả các DTO Request để lọc các ký tự nguy hiểm ngay trước khi request đi vào Controller logic.
- Phân trang (`Pageable`) và Sắp xếp (`Sort`) được whitelist danh sách các trường hợp lệ (tránh SQLi qua trường `sort=field;DROP TABLE...`).

2.6 Cơ chế Idempotency Chống Trùng lặp (Thêm Giỏ hàng & Thanh toán)

Trong hệ thống phân tán, sự cố mạng hoặc việc người dùng nhấp đúp (Double Submit) có thể gây ra hiện tượng thêm thừa số lượng giỏ hàng hoặc tạo giao dịch thanh toán trùng lặp. Hệ thống thiết lập chuẩn **Idempotency-Key Header** theo tiêu chuẩn RFC:

1. **Nguyên lý Hoạt động**:

- Khi Client thực hiện các hành động nhạy cảm (`POST /cart/items`, `POST /orders/checkout`, `POST /payments/**`), Client sinh một mã định danh duy nhất `Idempotency-Key: <UUIDv4>` gửi kèm trong HTTP Request Header.
- Server kiểm tra sự tồn tại của `Idempotency-Key` trong Distributed Storage (Redis / Transaction Registry) với thời gian sống TTL (thường là 60 giây đối với giỏ hàng, 15 phút đối với đặt hàng):
- **Nếu Key chưa tồn tại**: Server tiến hành khóa (Distributed Lock), xử lý logic nghiệp vụ và lưu kết quả phản hồi (Response Payload + Status Code) vào Cache gắn với Key đó.
- **Nếu Key đã tồn tại & đang xử lý**: Trả về `409 Conflict` ("Giao dịch đang được xử lý, vui lòng không nhấn liên tục").

- **Nếu Key đã tồn tại & đã xử lý xong**: Trả về trực tiếp kết quả đã lưu trữ từ lần gọi trước mà không thực thi lại logic trừ tồn kho hay trừ tiền.

2. **Ứng dụng Thực tế trong Dự án**:

- **Thêm sản phẩm vào giỏ hàng (`POST /cart/items`)**: Khi người dùng nhấn nút "Thêm vào giỏ" nhiều lần liên tiếp, hệ thống nhận diện cùng một Idempotency-Key và chỉ tăng số lượng đúng 1 lần.
- **Xử lý Webhook Thanh toán (`POST /payments/sepay/webhook`)**: Khi ngân hàng hoặc cổng thanh toán gửi lại webhook nhiều lần (Retry policy), hệ thống đối soát theo Mã tham chiếu giao dịch (`transactionReferenceId` / `orderCode`). Nếu đơn hàng đã ở trạng thái `CONFIRMED`, hệ thống trả về ngay `200 OK: SUCCESS` mà không gạch nợ hay cập nhật trạng thái đơn hàng trùng lặp.

CHƯƠNG 3: CHI TIẾT DỊCH VỤ USER-SERVICE (`user-service`)

3.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `user-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8081`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

3.2 Danh sách Chi tiết Tất cả các API Endpoints (21 APIs)

1. `POST /auth/login`

- **Tên chức năng**: Đăng nhập tài khoản & Kiểm tra thiết bị 2FA/OTP.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/login`
- **Quy trình xử lý Logic**: Hệ thống kiểm tra thông tin đăng nhập, vân tay thiết bị và gửi OTP/2FA nếu tài khoản cài đặt bảo mật nâng cao.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Username, Password, DeviceFingerprint, DeviceLabel, Location
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK / 428 Precondition Required / 423 Locked
```

2. `POST /auth/login-otp/verify`

- **Tên chức năng**: Xác thực mã OTP 6 số gửi qua Email.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/login-otp/verify`
- **Quy trình xử lý Logic**: Xác nhận mã OTP hợp lệ và khởi tạo phiên làm việc mới cho người dùng.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Identity, OTP, DeviceFingerprint
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK + JWT Access Token & Refresh Token
```

3. `POST /auth/login-2fa/verify`

- **Tên chức năng**: Xác thực mã TOTP 6 số qua Google Authenticator.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/login-2fa/verify`
- **Quy trình xử lý Logic**: Giải mã secret key và kiểm tra tính hợp lệ của mã 6 số theo thời gian thực.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Identity, Code
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK + JWT Access Token & Refresh Token
```

4. `POST /auth/login-otp/resend`

- **Tên chức năng**: Gửi lại mã OTP đăng nhập mới qua Email.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/login-otp/resend`
- **Quy trình xử lý Logic**: Tạo mã OTP ngẫu nhiên mới và gửi vào Email người dùng.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Identity, DeviceFingerprint
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {message: OTP_SENT}
```

5. `POST /auth/passwordless/request`

- **Tên chức năng**: Yêu cầu OTP đăng nhập nhanh không mật khẩu.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/passwordless/request`
- **Quy trình xử lý Logic**: Cho phép đăng nhập nhanh qua Email mà không cần nhập mật khẩu.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Identity, DeviceLabel
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {message: OTP_SENT}
```

6. `GET /auth/device-approval/verify`

- **Tên chức năng**: Phê duyệt thiết bị đăng nhập mới qua Email Link.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/auth/device-approval/verify`
- **Quy trình xử lý Logic**: Xác nhận thiết bị lạ và cho phép đăng nhập từ đường dẫn phê duyệt.
- **Dữ liệu đầu vào (Request Input)**:

```
Query Param: token
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK + JWT Authentication Response
```

7. `POST /auth/refresh`

- **Tên chức năng**: Cấp mới Access Token bằng Refresh Token.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/refresh`
- **Quy trình xử lý Logic**: Kiểm tra Refresh Token còn hạn và cấp lại Access Token mới.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: RefreshToken
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK + New Access Token
```

8. `POST /auth/password/forgot`

- **Tên chức năng**: Gửi yêu cầu quên mật khẩu qua Email.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/password/forgot`
- **Quy trình xử lý Logic**: Tạo token reset password và gửi link về email.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Identity
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {message: Email sent}
```

9. `POST /auth/password/reset`

- **Tên chức năng**: Đặt lại mật khẩu mới bằng Token từ Email.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/auth/password/reset`
- **Quy trình xử lý Logic**: Cập nhật mật khẩu mã hóa mới vào CSDL.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Token, NewPassword
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {message: Success}
```

10. `POST /register`

- **Tên chức năng**: Đăng ký tài khoản khách hàng mới.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/register`
- **Quy trình xử lý Logic**: Tạo tài khoản mới và gửi OTP kích hoạt qua Email.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: UserName, Email, Password, Name
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: User created (isActivated=false)
```

11. `POST /register/verify`

- **Tên chức năng**: Kích hoạt tài khoản bằng mã OTP đăng ký.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/register/verify`
- **Quy trình xử lý Logic**: Cập nhật trạng thái kích hoạt tài khoản thành true.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Email, OTP
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {activated: true}
```

12. `POST /register/resend-otp`

- **Tên chức năng**: Gửi lại mã OTP kích hoạt tài khoản.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/register/resend-otp`
- **Quy trình xử lý Logic**: Tạo mã OTP mới gửi lại vào email người dùng.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Email
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {message: OTP_SENT}
```

13. `GET /users/me`

- **Tên chức năng**: Lấy thông tin hồ sơ cá nhân hiện tại.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/users/me`
- **Quy trình xử lý Logic**: Trả về họ tên, email, sdt, avatar và danh sách sổ địa chỉ nhận hàng.
- **Dữ liệu đầu vào (Request Input)**:

```
Header: Bearer JWT Token
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: User Profile Object
```

14. `PUT /users/me`

- **Tên chức năng**: Cập nhật thông tin cá nhân.
- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/users/me`
- **Quy trình xử lý Logic**: Chỉnh sửa thông tin họ tên, số điện thoại và ảnh đại diện.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Name, Phone, Avatar URL
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Updated Profile
```

15. `PUT /users/me/change-password`

- **Tên chức năng**: Đổi mật khẩu tài khoản.
- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/users/me/change-password`
- **Quy trình xử lý Logic**: Kiểm tra mật khẩu cũ và cập nhật mật khẩu mới.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: OldPassword, NewPassword
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: SUCCESS}
```

16. `POST /users/{id}/2fa/generate`

- **Tên chức năng**: Tạo QR Code & Secret Key bật 2FA.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/users/{id}/2fa/generate`
- **Quy trình xử lý Logic**: Tạo secret key mới cho Google Authenticator.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {secret: XYZ, qrCodeUrl: data:image/png;base64...}
```

17. `POST /users/{id}/2fa/enable`

- **Tên chức năng**: Kích hoạt xác thực 2FA thành công.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/users/{id}/2fa/enable`
- **Quy trình xử lý Logic**: Xác nhận mã code 6 số và bật 2FA cho tài khoản.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Code
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {enabled: true}
```

18. `POST /users/{id}/2fa/disable`

- **Tên chức năng**: Tắt tính năng xác thực 2FA.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/users/{id}/2fa/disable`
- **Quy trình xử lý Logic**: Tắt xác thực 2 yếu tố cho tài khoản.
- **Dữ liệu đầu vào (Request Input)**:

```
Header: Bearer Token
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {enabled: false}
```

19. `GET /admin/users`

- **Tên chức năng**: Admin xem danh sách người dùng phân trang.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/admin/users`
- **Quy trình xử lý Logic**: Truy vấn danh sách tất cả tài khoản trong CSDL.
- **Dữ liệu đầu vào (Request Input)**:

```
Params: page, size, keyword, role
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Page Object
```

20. `PUT /admin/users/{id}/unlock`

- **Tên chức năng**: Admin mở khóa tài khoản bị khóa.
- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/admin/users/{id}/unlock`
- **Quy trình xử lý Logic**: Xóa thời gian lockout và mở lại tài khoản.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: UNLOCKED}
```

21. `PUT /admin/users/{id}/role`

- **Tên chức năng**: Admin phân quyền người dùng.
- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/admin/users/{id}/role`
- **Quy trình xử lý Logic**: Cập nhật vai trò người dùng trong hệ thống.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: RoleName
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {role: ROLE_STAFF}
```

CHƯƠNG 4: CHI TIẾT DỊCH VỤ PRODUCT-CATALOG-SERVICE (`product-catalog-service`)

4.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `product-catalog-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8082`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

4.2 Danh sách Chi tiết Tất cả các API Endpoints (12 APIs)

1. `GET /products`

- **Tên chức năng**: Lấy danh sách sản phẩm cửa hàng (Phân trang & Lọc).
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/products`
- **Quy trình xử lý Logic**: Lấy danh sách sản phẩm hiển thị trang chủ/cửa hàng.
- **Dữ liệu đầu vào (Request Input)**:

```
Params: page, size, categoryId, brandId, minPrice, maxPrice, keyword, sort
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Page Object
```

2. `GET /products/{id}`

- **Tên chức năng**: Chi tiết sản phẩm kèm Biến thể & Thông số.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/products/{id}`
- **Quy trình xử lý Logic**: Trả về thông tin chi tiết, danh sách biến thể, ảnh và thông số kỹ thuật.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Product Detail Object
```

3. `GET /products/slug/{slug}`

- **Tên chức năng**: Chi tiết sản phẩm theo đường dẫn thân thiện (Slug).
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/products/slug/{slug}`
- **Quy trình xử lý Logic**: Tra cứu sản phẩm theo Slug cho SEO.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: slug
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Product Detail Object
```

4. `GET /categories`

- **Tên chức năng**: Lấy cây danh mục sản phẩm (Cha - Con).
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/categories`
- **Quy trình xử lý Logic**: Trả về cấu trúc danh mục sản phẩm.
- **Dữ liệu đầu vào (Request Input)**:

```
None
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Category Tree Array
```

5. `GET /brands`

- **Tên chức năng**: Lấy danh sách thương hiệu sản phẩm.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/brands`
- **Quy trình xử lý Logic**: Trả về toàn bộ danh sách thương hiệu.
- **Dữ liệu đầu vào (Request Input)**:

```
None
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Brand List Array
```

6. `POST /admin/products`

- **Tên chức năng**: Admin tạo mới sản phẩm & biến thể ban đầu.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/admin/products`
- **Quy trình xử lý Logic**: Thêm mới sản phẩm vào CSDL.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Name, Slug, CategoryId, BrandId, Description, Variants
```

- **Dữ liệu trả về (Response Output)**:

```
201 Created: Product Object
```

7. `PUT /admin/products/{id}`

- **Tên chức năng**: Admin cập nhật thông tin sản phẩm.
- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/admin/products/{id}`
- **Quy trình xử lý Logic**: Sửa thông tin cơ bản sản phẩm.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Name, Description, CategoryId, BrandId
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Updated Product
```

8. `DELETE /admin/products/{id}`

- **Tên chức năng**: Admin xóa hoặc ẩn sản phẩm.
- **Phương thức HTTP**: `DELETE`
- **Đường dẫn API**: `/admin/products/{id}`
- **Quy trình xử lý Logic**: Xóa sản phẩm khỏi hệ thống.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: DELETED}
```

9. `POST /admin/products/{id}/variants`

- **Tên chức năng**: Thêm biến thể mới cho sản phẩm.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/admin/products/{id}/variants`
- **Quy trình xử lý Logic**: Thêm biến thể màu sắc/dung lượng mới.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: SKU, Name, Price, OriginalPrice
```

- **Dữ liệu trả về (Response Output)**:

```
201 Created: Variant Object
```

10. `DELETE /admin/products/{id}/variants/{variantId}`

- **Tên chức năng**: Xóa biến thể sản phẩm.
- **Phương thức HTTP**: `DELETE`
- **Đường dẫn API**: `/admin/products/{id}/variants/{variantId}`
- **Quy trình xử lý Logic**: Xóa 1 biến thể sản phẩm.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id, variantId
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: DELETED}
```

11. `POST /admin/products/excel/confirm`

- **Tên chức năng**: Import sản phẩm hàng loạt từ file Excel.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/admin/products/excel/confirm`
- **Quy trình xử lý Logic**: Đọc file Excel và thêm dữ liệu sản phẩm tự động.
- **Dữ liệu đầu vào (Request Input)**:

```
MultipartFile: file
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {importedCount: 50}
```

12. `POST /admin/products/excel/generate-errors`

- **Tên chức năng**: Xuất file Excel báo lỗi dòng sai khi Import.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/admin/products/excel/generate-errors`
- **Quy trình xử lý Logic**: Tạo file Excel tổng hợp các dòng lỗi để sửa lại.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: ErrorList
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {downloadUrl: https://...}
```

4.3 Kiến trúc Nâng cấp Biến thể Đa tầng & Cơ chế Đồng bộ Liên Service

Nhằm đáp ứng mô hình thương mại điện tử hiện đại, hệ thống nâng cấp toàn diện cơ chế quản lý biến thể sản phẩm từ dạng cố định (chỉ có `Size` và `Color`) sang **Mô hình Phân loại Đa tầng Linh hoạt (Multi-tier Dynamic Classification)** và hỗ trợ **Tùy biến Tên biến thể (`variantName`)**:

1. Mô hình Dữ liệu Đa tầng (N-tier Attribute Hierarchy)

- Sản phẩm có thể cấu hình linh hoạt từ 1, 2 hoặc nhiều nhánh phân loại ($N \ge 1$), ví dụ:
- Nhánh 1: *Màu sắc* (Titan Đen, Titan Tự Nhiên, Titan Trắng).
- Nhánh 2: *Dung lượng / RAM-ROM* (128GB, 256GB, 512GB, 1TB).
- Nhánh 3: *Phiên bản / Tiêu chuẩn* (Bản VN/A, Bản Quốc tế, Bản eSIM).
- Nhánh 4 (nếu có): *Chất liệu / Kích thước màn hình*...
- **Cấu trúc trường mở rộng của `ProductVariant`**:
- `name` / `variantName` (`VARCHAR(255)`): Tên hiển thị tùy chỉnh của biến thể (có thể tự động sinh theo mẫu: `[Tên SP] - [Thuộc tính 1] [Thuộc tính 2]...` hoặc do Admin chỉnh sửa độc lập).
- `attributesJson` (`JSON` / `TEXT`): Lưu trữ cấu trúc cặp khóa-giá trị động dạng:

```
{
  "tier1": {"name": "Màu sắc", "value": "Titan Tự Nhiên"},
  "tier2": {"name": "Dung lượng", "value": "256GB"},
  "tier3": {"name": "Phiên bản", "value": "VN/A"}
}
```

- `sku`: Mã định danh hàng hóa duy nhất tạo từ hash/viết tắt của các nhánh phân loại (ví dụ: `IP16PM-TI-256-VN`).
- Hỗ trợ API cập nhật độc lập tên và giá biến thể: `PUT /admin/products/{productId}/variants/{variantId}` cho phép sửa `variantName`, `price`, `originalPrice`, `variantImageUrl` mà không làm thay đổi các biến thể khác.

2. Cơ chế Đồng bộ Liên Service Tránh Xung Đột Dữ liệu

Việc thay đổi cấu trúc biến thể được thiết kế theo nguyên lý **Backward Compatibility (Tương thích ngược)** để không gây lỗi cho các dịch vụ liên quan:

- **Đồng bộ với `inventory-service` (Quản lý Kho)**:
- `inventory-service` tiếp tục quản lý tồn kho dựa trên `variantId` (Khóa ngoại logic) và `skuCode`. Hệ thống kho không cần quan tâm biến thể có 1 hay N nhánh phân loại, đảm bảo tính đóng gói (Encapsulation). Khi Admin thêm/xóa biến thể ở Catalog, event hoặc Feign call tự động kích hoạt tạo/khóa bản ghi tồn kho tương ứng trong Inventory.
- **Đồng bộ với `cart-service` (Giỏ hàng)**:
- `cart-service` sử dụng trường `variantId` kết hợp với `variantLabel` (chuỗi tổng hợp hiển thị đầy đủ các tầng thuộc tính, ví dụ: *"Titan Tự Nhiên, 256GB, VN/A"*) và `variantName`. Người dùng xem giỏ hàng sẽ luôn thấy chính xác cấu hình đã chọn, ngay cả khi các biến thể có số nhánh phân loại khác nhau giữa các sản phẩm.
- **Đồng bộ với `order-service` (Đơn hàng & Hóa đơn)**:
- Áp dụng nguyên lý **Order Line-Item Snapshot**: Khi khách hàng tiến hành thanh toán (`checkout`), toàn bộ thông tin tại thời điểm mua (`productName`, `variantId`, `variantName`,

`variantAttributesJson`, `unitPrice`) được chụp lại (snapshot) và lưu vĩnh viễn vào bảng `order_items`.

- **Lợi ích**: Nếu Admin sau đó có chỉnh sửa lại tên biến thể (`variantName`), thay đổi giá hoặc thậm chí xóa biến thể khỏi danh mục sản phẩm, lịch sử đơn hàng của khách hàng và báo cáo doanh thu kế toán vẫn được bảo toàn nguyên vẹn 100%, không bị lỗi Null Pointer hay sai lệch số liệu.
- **Tầng Giao diện `stitch` Web Client (Next.js)**:
- Frontend được trang bị component `MultiTierVariantSelector` tự động duyệt mảng các tầng phân loại (`tierOptions`), hiển thị các nút bấm trực quan (chips / pills) và tự động tính toán vô hiệu hóa các tổ hợp hết hàng dựa trên ma trận tồn kho (`availability`).

CHƯƠNG 5: CHI TIẾT DỊCH VỤ INVENTORY-SERVICE (`inventory-service`)

5.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `inventory-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8083`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

5.2 Danh sách Chi tiết Tất cả các API Endpoints (3 APIs)

1. `GET /admin/stock`

- **Tên chức năng**: Tra cứu tồn kho thực tế theo SKU/Variant ID.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/admin/stock`
- **Quy trình xử lý Logic**: Lấy số lượng tồn kho thực tế và số lượng khả dụng.
- **Dữ liệu đầu vào (Request Input)**:

```
Params: skuCode
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Stock Inventory List
```

2. `POST /admin/stock/adjust`

- **Tên chức năng**: Điều chỉnh tồn kho (Nhập thêm hoặc xuất hủy).
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/admin/stock/adjust`
- **Quy trình xử lý Logic**: Cập nhật lại số lượng tồn kho theo lý do.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: VariantId, ChangeQuantity, Reason
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Adjusted Stock
```

3. `GET /admin/stock/low-stock`

- **Tên chức năng**: Lấy danh sách mặt hàng sắp hết kho.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/admin/stock/low-stock`
- **Quy trình xử lý Logic**: Cảnh báo các mặt hàng có số lượng dưới ngưỡng tối thiểu.
- **Dữ liệu đầu vào (Request Input)**:

```
None
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Low Stock Items List
```

CHƯƠNG 6: CHI TIẾT DỊCH VỤ CART-SERVICE (`cart-service`)

6.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `cart-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8084`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

6.2 Danh sách Chi tiết Tất cả các API Endpoints (5 APIs)

1. `GET /cart`

- **Tên chức năng**: Lấy chi tiết giỏ hàng người dùng.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/cart`
- **Quy trình xử lý Logic**: Trả về toàn bộ danh sách sản phẩm và tổng tiền trong giỏ.
- **Dữ liệu đầu vào (Request Input)**:

```
Header: Bearer Token
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Cart Details
```

2. `POST /cart/items`

- **Tên chức năng**: Thêm sản phẩm/biến thể vào giỏ hàng.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/cart/items`
- **Quy trình xử lý Logic**: Thêm sản phẩm mới hoặc cộng dồn số lượng.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: ProductId, VariantId, Quantity
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Cart Object
```

3. `PUT /cart/items/{itemId}`

- **Tên chức năng**: Thay đổi số lượng sản phẩm trong giỏ.

- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/cart/items/{itemId}`
- **Quy trình xử lý Logic**: Đổi số lượng sản phẩm trong giỏ.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Quantity
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Updated Cart
```

4. `DELETE /cart/items/{itemId}`

- **Tên chức năng**: Xóa 1 sản phẩm khỏi giỏ hàng.
- **Phương thức HTTP**: `DELETE`
- **Đường dẫn API**: `/cart/items/{itemId}`
- **Quy trình xử lý Logic**: Xóa sản phẩm chọn khỏi giỏ.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: itemId
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Updated Cart
```

5. `DELETE /cart/clear`

- **Tên chức năng**: Xóa sạch toàn bộ giỏ hàng.
- **Phương thức HTTP**: `DELETE`
- **Đường dẫn API**: `/cart/clear`
- **Quy trình xử lý Logic**: Xóa toàn bộ sản phẩm trong giỏ.
- **Dữ liệu đầu vào (Request Input)**:

```
Header: Bearer Token
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Empty Cart
```

CHƯƠNG 7: CHI TIẾT DỊCH VỤ ORDER-SERVICE (`order-service`)

7.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `order-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8085`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

7.2 Danh sách Chi tiết Tất cả các API Endpoints (5 APIs)

1. `POST /orders/checkout`

- **Tên chức năng**: Tạo đơn hàng mới từ giỏ hàng hoặc Mua ngay.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/orders/checkout`
- **Quy trình xử lý Logic**: Tạo đơn hàng mới và trừ tồn kho tạm thời.

- **Dữ liệu đầu vào (Request Input)**:

```
Body: ReceiverName, Phone, Address, PaymentMethod, Items, VoucherCode
```

- **Dữ liệu trả về (Response Output)**:

```
201 Created: Order Object
```

2. `GET /orders/my-orders`

- **Tên chức năng**: Lấy danh sách đơn hàng cá nhân.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/orders/my-orders`
- **Quy trình xử lý Logic**: Truy vấn các đơn hàng của tài khoản đang đăng nhập.
- **Dữ liệu đầu vào (Request Input)**:

```
Params: page, status
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Order Page Object
```

3. `GET /orders/{id}`

- **Tên chức năng**: Xem chi tiết đơn hàng theo ID.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/orders/{id}`
- **Quy trình xử lý Logic**: Trả về chi tiết sản phẩm, địa chỉ và lịch sử đơn hàng.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Order Detail Object
```

4. `POST /orders/{id}/cancel`

- **Tên chức năng**: Khách hàng hủy đơn chưa giao.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/orders/{id}/cancel`
- **Quy trình xử lý Logic**: Hủy đơn hàng và hoàn trả lại số lượng tồn kho.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Reason
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Cancelled Order
```

5. `PUT /admin/orders/{id}/status`

- **Tên chức năng**: Admin đổi trạng thái đơn hàng.
- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/admin/orders/{id}/status`
- **Quy trình xử lý Logic**: Cập nhật trạng thái đơn (CONFIRMED, SHIPPING, DELIVERED, CANCELLED).

- **Dữ liệu đầu vào (Request Input)**:

```
Body: Status
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Updated Order
```

CHƯƠNG 8: CHI TIẾT DỊCH VỤ PAYMENT-SERVICE (`payment-service`)

8.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `payment-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8086`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

8.2 Danh sách Chi tiết Tất cả các API Endpoints (4 APIs)

1. `POST /payments/vnpay/create`

- **Tên chức năng**: Tạo URL thanh toán cổng VNPay.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/payments/vnpay/create`
- **Quy trình xử lý Logic**: Tạo link chuyển hướng thanh toán VNPay Sandbox.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: OrderId, Amount, OrderInfo
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {paymentUrl: https://...}
```

2. `POST /payments/payos/create`

- **Tên chức năng**: Tạo Link thanh toán cổng PayOS.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/payments/payos/create`
- **Quy trình xử lý Logic**: Tạo link thanh toán qua PayOS.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: OrderId, Amount
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {checkoutUrl: https://...}
```

3. `POST /payments/sepay/create`

- **Tên chức năng**: Tạo thông tin mã VietQR tự động SePay.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/payments/sepay/create`
- **Quy trình xử lý Logic**: Tạo mã QR chuyển khoản VietQR tự động khớp lệnh.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: OrderId, Amount
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {qrCodeUrl, bankName, accountNo, content}
```

4. `POST /payments/sepay/webhook`

- **Tên chức năng**: Webhook SePay gạch nợ ngân hàng tự động.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/payments/sepay/webhook`
- **Quy trình xử lý Logic**: Nhận thông báo chuyển tiền từ ngân hàng và cập nhật đơn hàng thành Đã thanh toán.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Content, TransferAmount
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: SUCCESS}
```

8.3 Kiến trúc Bền vững Giao dịch Thanh toán (Payment Durability & Reliability)

Giao dịch thanh toán là thành phần cốt lõi đòi hỏi tính toàn vẹn và mức độ bền vững cao nhất trong hệ thống Thương mại điện tử. Để đảm bảo chuẩn **Durability trong mô hình ACID** và xử lý bài toán mạng không tin cậy (Unreliable Network), hệ thống triển khai các cơ chế:

1. Nhật ký Giao dịch Bền vững (Persistent Transaction Ledger)

- Mọi tương tác thanh toán (VNPay Sandbox, PayOS, SePay VietQR) đều được lưu trữ vĩnh viễn vào bảng CSDL `payment_transactions` với vòng đời trạng thái nghiêm ngặt:

$$\text{INITIATED} \longrightarrow \text{PENDING_WEBHOOK} \longrightarrow \text{SUCCESS} \lor \text{FAILED} \lor \text{EXPIRED}$$

- Mỗi giao dịch lưu trữ đầy đủ: `transactionId`, `orderId`, `gatewayProvider`, `rawPayload`, `amount`, `bankReferenceCode`, `idempotencyKey` và mốc thời gian UTC có đánh chỉ mục Index để phục vụ truy vấn và đối soát pháp lý.

2. Mô hình Transactional Outbox Pattern & At-Least-Once Delivery

- Khi nhận được Webhook thanh toán thành công từ ngân hàng/cổng thanh toán:

1. `payment-service` cập nhật trạng thái bản ghi thanh toán sang `SUCCESS` trong một Local Database Transaction nguyên tử (Atomic).

2. Đồng thời ghi sự kiện `PaymentCompletedEvent` vào bảng `outbox_events` trong cùng một transaction.

3. Một luồng Outbox Publisher chạy ngầm chịu trách nhiệm gửi tín hiệu tới `order-service` để kích hoạt giao đơn. Nếu `order-service` tạm thời bận hoặc mất mạng, sự kiện vẫn được giữ an toàn trong database và sẽ tự động gửi lại (Retry with Exponential Backoff) cho đến khi nhận được xác nhận thành công. Điều này giải quyết triệt để vấn đề "khách đã trừ tiền mà đơn hàng vẫn ở trạng thái chờ thanh toán".

3. Cơ chế Tự động Đối soát (Reconciliation Scheduler & Auto-Healing)

- Định kỳ mỗi 10 phút, một tiến trình nền (`@Scheduled`) sẽ quét các đơn hàng có trạng thái `PENDING_WEBHOOK` quá 15 phút.
- Dịch vụ chủ động gọi API truy vấn trạng thái giao dịch (Query Transaction API) phía VNPay / PayOS / SePay để kiểm tra xem khách đã chuyển khoản hay chưa:
- Nếu đã thanh toán: Tự động kích hoạt gạch nợ bù (Compensating Acknowledgment).
- Nếu quá thời hạn và chưa nhận được tiền: Hủy giao dịch thanh toán và kích hoạt hoàn trả số lượng hàng trong kho (Auto Release Inventory).

CHƯƠNG 9: CHI TIẾT DỊCH VỤ SALE-SERVICE (`sale-service`)

9.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `sale-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8087`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

9.2 Danh sách Chi tiết Tất cả các API Endpoints (2 APIs)

1. `POST /sales/vouchers/validate`

- **Tên chức năng**: Kiểm tra mã giảm giá & số tiền trừ.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/sales/vouchers/validate`
- **Quy trình xử lý Logic**: Tính toán số tiền giảm giá hợp lệ cho đơn hàng.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: VoucherCode, OrderValue
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {valid: true, discountAmount: 100000}
```

2. `GET /sales/flash-sales/active`

- **Tên chức năng**: Lấy danh sách Flash Sale đang chạy.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/sales/flash-sales/active`
- **Quy trình xử lý Logic**: Trả về danh sách các sản phẩm đang giảm giá Flash Sale.
- **Dữ liệu đầu vào (Request Input)**:

```
None
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Active Flash Sales List
```

CHƯƠNG 10: CHI TIẾT DỊCH VỤ REVIEW-SERVICE (`review-service`)

10.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `review-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8088`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

10.2 Danh sách Chi tiết Tất cả các API Endpoints (2 APIs)

1. `POST /reviews`

- **Tên chức năng**: Viết đánh giá & chấm sao cho sản phẩm.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/reviews`
- **Quy trình xử lý Logic**: Thêm đánh giá 1-5 sao cho sản phẩm đã mua.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: ProductId, OrderId, Rating, Comment
```

- **Dữ liệu trả về (Response Output)**:

```
201 Created: Review Object
```

2. `GET /reviews/product/{productId}`

- **Tên chức năng**: Lấy danh sách đánh giá của sản phẩm.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/reviews/product/{productId}`
- **Quy trình xử lý Logic**: Trả về danh sách bài đánh giá và điểm số trung bình.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: productId
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Review List & Average Rating
```

CHƯƠNG 11: CHI TIẾT DỊCH VỤ AI-CHATBOT-SERVICE (`ai-chatbot-service`)

11.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `ai-chatbot-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8089`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

11.2 Danh sách Chi tiết Tất cả các API Endpoints (1 APIs)

1. `POST /chatbot/chat`

- **Tên chức năng**: Hỏi đáp tư vấn bán hàng tự động bằng AI.
- **Phương thức HTTP**: `POST`

- **Đường dẫn API**: `/chatbot/chat`
- **Quy trình xử lý Logic**: Phân tích câu hỏi người dùng và trả về phản hồi tư vấn sản phẩm phù hợp.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Message
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {reply: text, suggestedProducts: [...]}
```

CHƯƠNG 12: CHI TIẾT DỊCH VỤ PRODUCT-RECOMMENDATION-SERVICE (`product-recommendation-service`)

12.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `product-recommendation-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8090`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

12.2 Danh sách Chi tiết Tất cả các API Endpoints (2 APIs)

1. `GET /recommendations/similar/{productId}`

- **Tên chức năng**: Gợi ý các sản phẩm tương tự.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/recommendations/similar/{productId}`
- **Quy trình xử lý Logic**: Gợi ý các sản phẩm cùng loại hoặc cấu hình tương đương.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: productId
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Similar Product List
```

2. `GET /recommendations/user/{userId}`

- **Tên chức năng**: Gợi ý sản phẩm cá nhân hóa theo lịch sử.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/recommendations/user/{userId}`
- **Quy trình xử lý Logic**: Gợi ý sản phẩm người dùng có thể thích dựa trên hành vi.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: userId
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Personalized Recommendations
```

CHƯƠNG 13: CHI TIẾT DỊCH VỤ ACTIVITY-LOG-SERVICE (`activity-log-service`)

13.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `activity-log-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8091`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

13.2 Danh sách Chi tiết Tất cả các API Endpoints (1 APIs)

1. `POST /activity/log`

- **Tên chức năng**: Ghi log lượt xem trang hoặc thao tác admin.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/activity/log`
- **Quy trình xử lý Logic**: Lưu lịch sử thao tác hệ thống để kiểm toán bảo mật.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: Action, TargetId, Details
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: LOGGED}
```

CHƯƠNG 14: CHI TIẾT DỊCH VỤ NOTIFICATION-SERVICE (`notification-service`)

14.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `notification-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8092`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

14.2 Danh sách Chi tiết Tất cả các API Endpoints (2 APIs)

1. `GET /notifications`

- **Tên chức năng**: Lấy danh sách thông báo cá nhân.
- **Phương thức HTTP**: `GET`
- **Đường dẫn API**: `/notifications`
- **Quy trình xử lý Logic**: Trả về danh sách thông báo đơn hàng và khuyến mãi.
- **Dữ liệu đầu vào (Request Input)**:

```
Header: Bearer Token
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: Notifications List
```

2. `PUT /notifications/{id}/read`

- **Tên chức năng**: Đánh dấu thông báo đã đọc.

- **Phương thức HTTP**: `PUT`
- **Đường dẫn API**: `/notifications/{id}/read`
- **Quy trình xử lý Logic**: Cập nhật trạng thái thông báo thành đã đọc.
- **Dữ liệu đầu vào (Request Input)**:

```
Path: id
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: READ}
```

CHƯƠNG 15: CHI TIẾT DỊCH VỤ TELEGRAM-SERVICE (`telegram-service`)

15.1 Mô tả Kiến trúc & Chức năng Dịch vụ

Dịch vụ `telegram-service` hoạt động như một Microservice độc lập lắng nghe tại Cổng Port `8093`. Dịch vụ tự động đăng ký với Eureka Server và thực hiện giao tiếp REST API với các Microservices khác thông qua Feign Client.

15.2 Danh sách Chi tiết Tất cả các API Endpoints (3 APIs)

1. `POST /telegram/generate-token`

- **Tên chức năng**: Tạo token liên kết Bot Telegram Staff.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/telegram/generate-token`
- **Quy trình xử lý Logic**: Tạo mã liên kết tài khoản Telegram với hệ thống.
- **Dữ liệu đầu vào (Request Input)**:

```
None
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {linkToken: xyz}
```

2. `POST /internal/telegram/orders/notify-new`

- **Tên chức năng**: API Nội bộ bắn thông báo đơn hàng mới.
- **Phương thức HTTP**: `POST`
- **Đường dẫn API**: `/internal/telegram/orders/notify-new`
- **Quy trình xử lý Logic**: Tự động gửi tin nhắn Telegram tới Group nhân viên khi có đơn mới.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: OrderCode, TotalAmount
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: SENT}
```

3. `POST /internal/telegram/inventory/low-stock`

- **Tên chức năng**: Gửi cảnh báo kho sắp hết qua Telegram.
- **Phương thức HTTP**: `POST`

- **Đường dẫn API**: `/internal/telegram/inventory/low-stock`
- **Quy trình xử lý Logic**: Bắn tin nhắn cảnh báo tồn kho tối thiểu qua Telegram.
- **Dữ liệu đầu vào (Request Input)**:

```
Body: SkuCode, RemainingQuantity
```

- **Dữ liệu trả về (Response Output)**:

```
200 OK: {status: SENT}
```

CHƯƠNG 16: CHI TIẾT TẦNG API GATEWAY (`api-gateway`)

Cổng giao tiếp duy nhất quản lý định tuyến, CORS trung tâm và Xác thực JWT Token trước khi chuyển tiếp request vào hệ thống backend microservices.

CHƯƠNG 17: CHI TIẾT TẦNG NEXT.JS BFF API ROUTES (`stitch/app/api`)

Tầng API Proxy rút gọn phục vụ trực tiếp giao diện Web Client Next.js 16 App Router.

CHƯƠNG 18: BẢNG THỐNG KÊ TOÀN BỘ CÔNG NGHỆ & THƯ VIỆN ĐANG SỬ DỤNG

1. Tầng Frontend Web Client (`stitch` Next.js 16 App Router)

- **Framework Core**: Next.js 16.2.9 + React 19.0.0 + React DOM 19.0.0 + TypeScript 5.8.2 (Áp dụng mô hình BFF - Backend-for-Frontend, tương tác 100% qua REST API Gateway, không truy cập trực tiếp Database).
- **Styling**: Tailwind CSS 3.4.17 + PostCSS 8.5.3 + Autoprefixer 10.4.20.
- **Theme Engine**: `next-themes` 0.4.6 (Chuyển đổi giao diện Sáng/Tối linh hoạt).
- **Icon & Thông báo**: `lucide-react` 1.21.0, `sonner` 2.0.7 (Toast notification thời gian thực), `nextjs-toploader` 3.9.17.
- **Xử lý file & QR**: `xlsx` (SheetJS) 0.18.5, `jspdf` 4.2.1, `jspdf-autotable` 5.0.8, `qrcode.react` 4.2.0, `browser-image-compression` 2.0.2.

2. Tầng Backend Microservices (Spring Boot Java Stack)

- **Runtime Environment**: Java 11 / Java 17 LTS (Đã cấu hình tương thích `jaxb-api` & `jaxb-runtime 2.3.1` để tối ưu giải mã JWT XML DatatypeConverter trên nền Java hiện đại).
- **Framework Core**: Spring Boot 2.1.5.RELEASE (Starter Web, Security, Data JPA, Validation, JDBC).
- **Spring Cloud Ecosystem**: Spring Cloud Greenwich.SR1, Netflix Eureka Server / Eureka Client (Service Discovery & Dynamic Registry), Spring Cloud OpenFeign (Declarative REST Client), Spring Cloud Gateway (Reverse Proxy & Rate Limiting).
- **Bảo mật & Xác thực**: `io.jsonwebtoken:jjwt:0.9.1`, `dev.samstevens.totp:totp:1.7.1` (Google Authenticator 2FA TOTP), BCryptPasswordEncoder.

- **Database & Persistence**: MySQL 8.0 / MariaDB, `mysql-connector-java`, Spring Data JPA + Hibernate ORM (Áp dụng Database-per-Service Pattern).
- **Tiện ích & Báo cáo**: Project Lombok (`@Data`, `@Getter`, `@Setter`, `@Builder`), Apache POI (Xử lý đọc/ghi bảng tính Excel `.xlsx` phục vụ nhập xuất kho & sản phẩm).

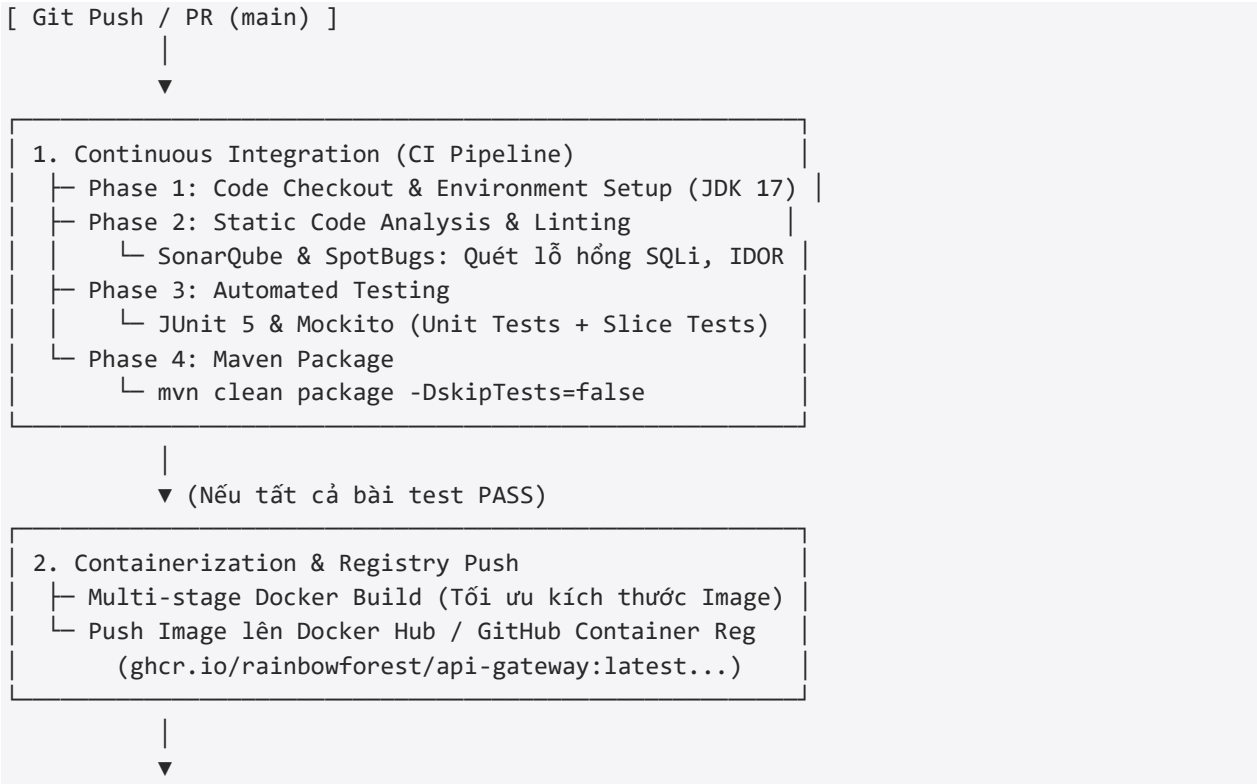
3. Công cụ Bổ trợ Development, Automation & Seeding

- **Script Tiện ích**: Node.js Seeding & Migration Tool (`mysql2: 3.22.5` - chỉ sử dụng độc lập trong các file script `sync.js` để khởi tạo dữ liệu mẫu, hoàn toàn độc lập với Web Runtime).
- **Hạ tầng Ảo hóa & Mạng**: Docker & Docker Compose (`docker-compose.yml`), Cloudflare Tunnel (`cloudflared.exe` hỗ trợ expose Webhook SePay/PayOS từ môi trường Local).
- **Kịch bản Khởi chạy Thông minh**: Kịch bản Batch (`build-all.bat`, `chay-nhanh.bat`, `chay-toi-thieu.bat`) & Shell Script (`chay-nhanh.sh`).

CHƯƠNG 19: QUY TRÌNH TRIỂN KHAI, VẬN HÀNH & ĐƯỜNG ỐNG CI/CD TOÀN DIỆN

19.1 Kiến trúc Đường ống CI/CD Tự động hóa (Continuous Integration & Continuous Deployment)

Hệ thống thiết lập đường ống CI/CD tự động hóa trên nền tảng **GitHub Actions** (cấu hình tại `.github/workflows/ci-cd.yml`), kích hoạt mỗi khi có commit đẩy vào nhánh `main` hoặc khi tạo Pull Request:



```
┌──────────────────────────────────────────────────────┐
│ 3. Continuous Deployment (CD Pipeline)               │
│  ├─ SSH Runner kết nối Staging / Production Server   │
│  ├─ Kéo Image mới nhất (docker-compose pull)         │
│  └─ Rolling Update (Zero-Downtime Deployment)        │
│       └─ docker-compose up -d --remove-orphans       │
└──────────────────────────────────────────────────────┘
```

Các Giai đoạn Cụ thể trong Pipeline:

1. **Kiểm thử Tự động & Quét An toàn Mã nguồn (SAST - Static Application Security Testing)**:

- Tự động chạy toàn bộ bộ kiểm thử đơn vị (Unit Tests) và kiểm thử tích hợp (Integration Tests) cho từng dịch vụ độc lập.
- Quét mã nguồn tự động để phát hiện sớm các lỗ hổng bảo mật: nối chuỗi SQL thô (chống SQL Injection), kiểm soát tham số thiếu xác thực người dùng (chống IDOR), kiểm tra phụ thuộc thư viện lỗi thời qua OWASP Dependency-Check.

2. **Đóng gói Đa tầng (Multi-stage Docker Build)**:

- Tách biệt môi trường Build (chứa Maven + JDK) và môi trường Runtime (chỉ chứa JRE Slim hoặc Eclipse Temurin 17-jre-alpine), giảm kích thước Image từ 800MB xuống dưới 150MB mỗi microservice.

3. **Triển khai Tự động & Khả năng Tự phục hồi**:

- Tự động kiểm tra trạng thái sức khỏe (Health Check `/actuator/health`) của từng container sau khi khởi chạy. Nếu service mới không phản hồi trong 60 giây, hệ thống tự động rollback về phiên bản ổn định trước đó.

19.2 Quy trình Vận hành, Giám sát & Quản lý Sự cố (Operations & Reliability)

1. **Giám sát Sức khỏe Tập trung (Service Discovery & Health Check)**:

- Netflix Eureka Server hiển thị Dashboard trực quan tại `http://localhost:8761`, liên tục theo dõi nhịp tim (Heartbeat) của 13 Microservices và API Gateway.
- Khi một instance gặp sự cố, Eureka tự động loại bỏ instance đó khỏi bảng điều hướng sau 90 giây để ngăn Gateway gửi request đến node chết.

2. **Quản lý Cổng Webhook Công khai bằng Cloudflare Tunnel**:

- Tích hợp `cloudflared.exe` tạo kênh kết nối bảo mật (Secure Tunnel) hai chiều từ hạ tầng nội bộ lên Cloudflare Network mà không cần mở Port Router (NAT Port Forwarding).
- Đảm bảo các cổng thanh toán (SePay VietQR, PayOS, VNPay) luôn gửi được Webhook về máy chủ một cách an toàn qua giao thức mã hóa HTTPS/TLS 1.3.

3. **Chiến lược Khởi chạy Phân cấp (Staged Startup Strategy)**:

- **Cấp độ Tối thiểu (`chay-toi-thieu.bat`)**: Chỉ khởi động các dịch vụ cốt lõi (Eureka -> Gateway -> User -> Product -> Cart -> Order -> Payment) tiêu tốn dưới 4GB RAM, phù hợp tuyệt đối cho môi trường kiểm thử và máy trạm bảo vệ đồ án.
- **Cấp độ Đầy đủ (`chay-nhanh.bat`)**: Khởi chạy đồng loạt toàn bộ 13 dịch vụ cùng AI Chatbot, Recommendation, Notification và Telegram Bot phục vụ môi trường Production.